

Alfabetos, cadenas y lenguajes

Definición (Alfabeto)

Diremos que un conjunto finito Σ es un *alfabeto* si $\Sigma \neq \emptyset$ y $\forall x \in \Sigma$, x es un símbolo indivisible (no está compuesto por subsímbolos).

Definición (Cadena)

Una cadena o palabra w sobre un alfabeto Σ es una secuencia finita de símbolos de Σ .

Escribiremos a w de la forma $w = w_1 w_2 \dots w_k$, con $k \geq 0$, $w_i \in \Sigma$ y $0 \leq i \leq k$. Cuando $k = 0$ la cadena w se denomina cadena nula o palabra vacía y se denota con el símbolo λ . k es entonces la longitud de la cadena y se nota como $long(w) = |w| = k$ y representa el número de símbolos de Σ presentes en w contando los símbolos repetidos.

En algunos libros también se llama cardinalidad a la longitud de una palabra.

Definición (Concatenación)

Definiremos la *concatenación* de dos cadenas como una operación binaria, tal que, dadas dos cadenas x e y definidas sobre un alfabeto Σ , $x \bullet y = xy$. Formalmente, se define w como el resultado de concatenar x e y si $|w| = |x| + |y|$ y w es tal que $w_i = x_i$, $i = 1 \dots |x|$ y $w_{|x|+j} = y_j$ para $j = 1 \dots |y|$.

Ejercicio: dar una definición inductiva de la concatenación.

Definición (Σ^*)

Se define Σ^* al conjunto de todas las cadenas de longitud finita formadas por la concatenación de elementos de Σ . De esta manera, la cadena de longitud 0 o λ también se incluye en Σ^* .

Teorema

Sea Σ un alfabeto,

1. La concatenación es asociativa, es decir, para todo $x, y, z \in \Sigma^*$, $x \bullet (y \bullet z) = (x \bullet y) \bullet z$.
2. $[\Sigma, \bullet]$ es un monoide bajo concatenación (es decir tiene definida sobre ella una operación binaria y posee un elemento neutro) y λ es el elemento identidad, es decir, $x \in \Sigma^*$, $x \bullet \lambda = \lambda \bullet x = x$.

Las pruebas se dejan como ejercicio.

Definición (Potencia i -ésima de w)

Sea $w \in \Sigma^*$ y un número natural i , definimos la *potencia i -ésima* de la cadena w , notada como w^i recursivamente de la siguiente manera:

$$\begin{aligned}w^0 &= \lambda \\w^{(i+1)} &= w \bullet w^i \quad \forall i > 0\end{aligned}$$

Definición (Reverso)

Sea $w \in \Sigma^*$ con $w = w_1 w_2 \dots w_{n-1} w_n$ denominaremos *reverso* de la cadena w y lo notaremos w^R a la cadena $w^R = w_n w_{n-1} \dots w_2 w_1$. El reverso de una cadena se puede definir recursivamente de la siguiente manera:

$$w^R = \lambda \text{ si } w = \lambda (|w| = 0)$$

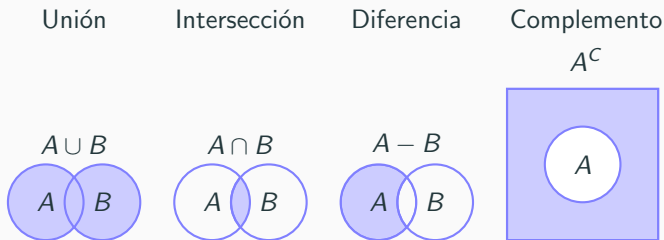
$$w^R = w_n \bullet w_r \text{ si } |w| = n + 1 > 0 \text{ y } w = w_r \bullet w_n, w_n \in \Sigma, w_r \in \Sigma^*$$

Definición (Lenguajes)

Denominaremos *Lenguaje* L definido sobre un alfabeto Σ a todo subconjunto de Σ^* .

Operaciones sobre lenguajes

Considerando que los lenguajes son conjuntos es posible definir sobre ellos las habituales operaciones entre lenguajes:



Además se definirán tres operaciones propias de los lenguajes:

- Concatenación,
- Potencia,
- Estrella de Kleene.

Definición (Concatenación de lenguajes)

Sean L_1 y L_2 dos lenguajes definidos sobre el alfabeto Σ , su **concatenación** L es un nuevo lenguaje obtenido mediante:

$$L = L_1 \bullet L_2 = \{w \mid w = x \bullet y \text{ para } x \in L_1, y \in L_2\}$$

Se utiliza el mismo símbolo $\bullet$ para lenguajes y cadenas para simplificar aunque se debe tener especial cuidado en su uso.

Definición (Potencia de lenguajes)

Sea L un lenguaje $\subseteq \Sigma^*$, definimos la **potencia de lenguajes** de la siguiente manera:

$$\begin{aligned} L^0 &= \{\lambda\} \\ L^{(i+1)} &= L \bullet L^i \quad \forall i \geq 0 \end{aligned}$$

Definición (Estrella de Kleene)

Sea L un lenguaje $\subseteq \Sigma^*$, definimos la operación **Estrella de Kleene o Clausura de Kleene** y la notaremos como L^* de la siguiente manera:

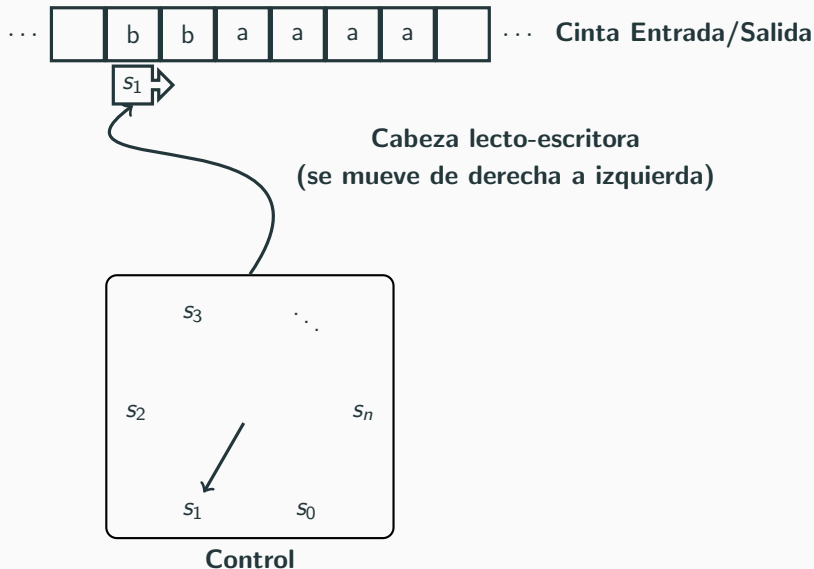
$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Definición (L^+)

Se define L^+ al conjunto $\bigcup_{i=1}^{\infty} L^i$, es decir, $L^+ = L \bullet L^*$. Es importante observar con atención que $\lambda \in L^+ \Leftrightarrow \lambda \in L$.

Para el caso de los alfabetos $\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i = \Sigma \bullet \Sigma^* = \Sigma - \{\lambda\}$.

Autómatas Finitos



Autómatas Finitos

Reconocedores

Definición (Autómata Finito Reconocedor)

Un *Autómata Finito Reconocedor* (AFR) es una quintupla $M = (S, \Sigma, \delta, s_0, F)$ donde:

- S es un conjunto de estados, $S \neq \emptyset$,
- Σ es el alfabeto de entrada,
- $\delta : S \times \Sigma \rightarrow S$ es la función de próximo estado o transición,
- s_0 es el estado inicial, $s_0 \in S$
- F es el conjunto de estados aceptadores o finales ($F \subseteq S$). F podría ser $\emptyset$ o todo S .

Definición

Un grafo dirigido es una terna (N, A, V) donde N es el conjunto de todos los vértices o nodos, A es el conjunto de arcos y V es la función que asocia a cada arco $a \in A$ con un par ordenado de vértices. En particular para los AF utilizaremos multidigraphos y los llamaremos *Diagramas de Estados*.

- Existirá un nodo $n \in N$ del grafo por cada estado $s \in S$
- Existirá un arco $a \in A$ tal que $V(a) = (s_i, s_k)$ si y solo si $\delta(s_i, a) = s_k$, donde $s_i, s_k \in S$ y $a \in A$. Tenga en cuenta que cada arco puede tener más de un rótulo.
- El estado inicial se indicará con el símbolo $\rightarrow$ y los estados aceptadores o finales con doble círculo.

Ejemplo

Supongamos que queremos hacer un AFR que dada una cadena de entrada decida si la misma es un identificador válido de *Python* sin tener en cuenta su longitud.

Ejemplo

Supongamos que queremos hacer un AFR que dada una cadena de entrada decida si la misma cumple con ser un password válido teniendo en cuenta que: El password debe tener **como máximo** 5 caracteres y al menos uno de ellos debe ser una letra en mayúscula. El resto de los caracteres pueden ser letras ($a \dots z, A \dots Z$) o dígitos ($0 \dots 9$).

Pensemos en las cadenas que serán aceptadas por M , por ejemplo: *Arya*, *JON*, *s4Nsa,gH0st* y muchas otras. En tanto que *edd*, *jorah*, *nightking* serán rechazadas.

Lema

Dado un autómata finito que permite transiciones etiquetadas con λ , se puede construir otro autómata equivalente sin transiciones etiquetadas con λ .

Demostración.

Vamos a dar simplemente un esquema de esta demostración. Dado un AF M puede obtenerse otro M' no determinista equivalente y sin transiciones λ a través de las siguientes transformaciones:

1. Agregar a M' una λ -transición de un estado s_i a otro s_j cuando en M se llegaba desde s_i a s_j por medio de más de una transición λ .
2. Poner como estados aceptadores todos aquellos estados de M que por λ permiten llegar a un estado aceptador.
3. por cada par de transiciones $\delta(s_i, \lambda) = s_j$, $\delta(s_j, a) = s_k$ agregar $\delta(s_i, a) = s_k$.
4. Sacar del autómata obtenido hasta el momento las λ -transiciones $\delta(s_i, \lambda) = s_j$ para todo $s_i, s_j \in S$.

□

Autómata Finito Reconocedor (con λ)

Definición

Un *autómata finito reconocedor* (AFR) es una quintupla $M = (S, \Sigma, \delta, s_0, F)$ donde:

- S es el conjunto de estados, $S \neq \emptyset$.
- Σ es el alfabeto de entrada.
- $\delta : S \times (\Sigma \cup \{\lambda\}) \rightarrow S$ es la función de próximo estado o de transición.
- s_0 es el estado inicial, $s_0 \in S$.
- F es el conjunto de estados finales o aceptadores $F \subseteq S$.

Esta será la definición que utilizaremos de aquí en más para los AFR.

Definición

Sea Σ un alfabeto, se dice que un AFR $M = (S, \Sigma, \delta, s_0, F)$ acepta (o reconoce) una cadena $w \in \Sigma^*$ si, partiendo de $\delta(s_0, a)$, alcanzamos un estado s_f donde $s_f \in F$.

Definición

Sea $M = (S, \Sigma, \delta, s_0, F)$ un AFR, el lenguaje reconocido por el autómata M es $L(M) = \{w | w \text{ es aceptada o reconocida por } M\}$.